The Unitary Compiler Collection (UCC)

Exact Classical Simulation via the Factored State Architecture

Unitary Foundation Team

March 10, 2026

Agenda

1. The Simulation Bottleneck & The Pivot
2. The Core Theory: The Factored State
3. Walkthrough: A Toy Example
4. Software Architecture
5. Benchmarks & Results
6. Next Steps & Discussion

The Simulation Bottleneck & The Pivot

The Universal Simulation Bottleneck

Simulating fault-tolerant quantum circuits (FTQC) requires deep non-Clifford logic (e.g., T-gates). Standard exact simulators inevitably hit exponential walls:

- **Statevector Simulators (e.g., Qiskit):** Hit the $\mathcal{O}(2^N)$ Physical Memory Wall. Cannot exceed ~40-50 qubits regardless of circuit structure.
- **Stabilizer Rank (e.g., tsim):** Hit the $\mathcal{O}(2^{\alpha t})$ T-Count Wall. Branches on every T-gate; times out on deep arithmetic logic.
- **Generalized Stabilizers (e.g., SOFT):** Avoids the T-count wall, but dynamic tableau pivoting across sparse vectors introduces heavy $\mathcal{O}(n^2)$ overhead per shot.

Motivating Example: Magic State Cultivation (MSC)

The Goal: Evaluate Magic State Cultivation protocols over 10^{12} shots to escape the physical noise floor.

The Challenge

- Highly probabilistic (requires fast-fail post-selection).
- Deep non-Clifford T-gate entanglement.
- Standard tools simply cannot scale to trillions of shots at FTQC hardware scales without massive GPU clusters.

The “New” UCC: A Strategic Pivot

Previous UCC: An open-source routing and bridging compiler.
New UCC: A **Multi-Level Compiler** and **Schrödinger Virtual Machine (SVM)** built explicitly for the Early FTQC Era.

- We break the exponential bottlenecks by decoupling deterministic routing from probabilistic non-Clifford entanglement.
- UCC scales exponentially *only* with the **Peak Active Rank** ($k_{\max}$).

The Core Theory: The Factored State

Circuit Composition & The Heisenberg Mapping

We partition all quantum operations into two sets:

1. **Deterministic:** Clifford Unitaries ($C \in \mathcal{C}_n$).
2. **Active:** Non-Clifford rotations (T), Measurements, Pauli Noise.

Instead of applying Cliffords to the statevector, we map Active operations *backward* through the Clifford frame (U_C) into a static virtual basis:

$$\tilde{P}_O = (U_C^{(t)})^\dagger P_O U_C^{(t)}$$

Insight: Coordinate mapping is defined entirely by Pauli tracking, completely independent of complex amplitudes!

The Factored State Equation

We define the exact physical quantum state at time t via a strict factorization:

$$|\psi^{(t)}\rangle = \gamma^{(t)} U_C^{(t)} \tilde{P}^{(t)} \left(|\phi^{(t)}\rangle_A \otimes |0\rangle_D \right)$$

- $U_C^{(t)}$ **(Clifford Frame):** Deterministic geometric coordinate mapping.
- $\tilde{P}^{(t)}$ **(Virtual Pauli Frame):** Tracks discrete stochastic errors via virtual X and Z vectors: $\tilde{P}^{(t)} = \bigotimes_{j=0}^{n-1} X_j^{x_j^{(t)}} Z_j^{z_j^{(t)}}$. Evaluated via $\mathcal{O}(1)$ bitwise XORs.
- $|\phi^{(t)}\rangle_A$ **(Active Statevector):** A dense 2^k array tracking non-Clifford superposition on k *Active* virtual qubits.
- $|0\rangle_D$ **(Dormant Qubits):** The $n-k$ qubits mathematically locked in $|0\rangle$.
- $\gamma^{(t)}$ **(Global Scalar):** Tracks continuous phases and deferred normalization.

Virtual Compression (The "Secret Sauce")

How do we apply a mapped multi-qubit operator (e.g., $\tilde{P} = X_0 \otimes Z_1$) without evaluating a full 2^n statevector?

Lemma 2: Exact Pauli Compression

The Multi-Level compiler constructively synthesizes a sequence of virtual Cliffords V to compress *any* multi-qubit Pauli onto a **single virtual axis** v. ($V\tilde{P}V^\dagger = P_v$).

Lemma 1: Zero-Cost Dormancy

If V uses a Dormant qubit as a control (e.g., $\text{CNOT}_{D\to A}$), it mathematically acts as the Identity on the Active statevector. **Zero FLOPs required.**

Applying the Compression: Where does V go?

If we apply virtual Cliffords V to compress an operator $O(\tilde{P})$, how is the global state preserved?

The compiler mathematically inserts the identity $V^\dagger V = I$ into the factored state, effectively **pushing it forward**:

$$\ldots \underbrace{U_C^{(t)} V^\dagger}_{\text{1. Offline Frame}} \underbrace{\left[V O(\tilde{P}) V^\dagger \right]}_{\text{2. Localized Op}} \underbrace{\left(V \tilde{P}^{(t)} V^\dagger \right)}_{\text{3. Pauli Tracker}} \underbrace{V \left(|\phi^{(t)}\rangle_A \otimes |0\rangle_D \right)}_{\text{4. Active Array}}$$

The transformation V distributes exactly where it is needed:

1. **Clifford Frame:** Absorbs the inverse offline ($U_C \leftarrow U_C V^\dagger$).
2. **The Operation:** Compresses to act on a single virtual axis ($O(P_v)$).
3. **Pauli Frame:** Is dynamically conjugated via $\mathcal{O}(1)$ bitwise XORs.
4. **Active Array:** Undergoes the transformation V (which costs **zero FLOPs** if

Walkthrough: A Toy Example

Walkthrough: Step-by-Step Factored State

Circuit (2 Qubits)

$\rightarrow$ Init $|00\rangle$

Phys $H(q_0)$

Phys $T(q_0)$

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

System State at t

$U_C = I, \quad \tilde{P}^{(t)} = I$

$A = \emptyset, \quad D = \{0, 1\} \quad (\mathbf{k} = \mathbf{0})$

$|\phi\rangle_A = [1], \quad \gamma = 1.0$

Walkthrough: Step-by-Step Factored State

Circuit (2 Qubits)

Init $|00\rangle$

$\rightarrow$ Phys $H(q_0)$

Phys $T(q_0)$

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

System State at t

Action: Update deterministic frame offline.

$U_C = H_0$

$\tilde{P}^{(t)} = I$

$A = \emptyset, \quad D = \{0, 1\} \quad (\mathbf{k} = \mathbf{0})$

$|\phi\rangle_A = [1]$ *(Zero FLOPs!)*

Walkthrough: Step-by-Step Factored State

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

$\rightarrow$ Phys $T(q_0)$ *(Compile)*

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

Why no I/Z branching?

$T \propto \exp\left(-i\frac{\pi}{8}Z\right)$. *Because I is invariant, V distributes directly into the exponent:* $\exp\left(-i\frac{\pi}{8}VZ_0V^\dagger\right)$.

System State at t

Compile T-Gate (Rotation about Z):

$T \propto \exp\left(-i\frac{\pi}{8}\mathbf{Z_0}\right)$. We track the generator!

We don't branch into I and Z universes! Physical Z_0 maps through U_C to virtual $\mathbf{X_0}$.

Compiler applies $V = H_0$ to align $X_0 \rightarrow Z_0$ for the continuous diagonal phase.

Walkthrough: Step-by-Step Factored State

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

$\rightarrow$ Phys $T(q_0)$ *(Execute)*

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

System State at t

Execute T-Gate (Dimensionality Expansion):

Virtual $V^\dagger$ pushes into U_C ($U_C \leftarrow U_C \cdot H_0 = I$).

Axis q_0 activates.

$A = \{0\}, \quad D = \{1\} \quad (\mathbf{k} = \mathbf{1})$

$|\phi\rangle_A = [1, e^{i\pi/4}]^T$ *(Array Doubles!)*

$\gamma = e^{i\pi/8}/\sqrt{2}$ *(Absorbs global phase)*

Walkthrough: Step-by-Step Factored State

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

Phys $T(q_0)$

$\rightarrow$ Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

Key Insight:

The physical CX entangles the qubits, but because it is a Clifford, the Active Array doesn't care! It costs zero FLOPs.

System State at t

Physical Entanglement:

Clifford gate maps directly into the offline frame.

$$U_C \leftarrow \mathrm{CX}_{0\rightarrow 1} \cdot U_C = \mathrm{CX}_{0\rightarrow 1}$$

$$A = \{0\}, \quad D = \{1\} \quad (\mathbf{k} = \mathbf{1})$$

$$|\phi\rangle_A = [1, e^{i\pi/4}]^T$$

The array is completely untouched!

Walkthrough: Step-by-Step Factored State

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

Phys $T(q_0)$

Phys $\mathrm{CX}(q_0 \to q_1)$

$\to$ Measure q_1

System State at t

Action: Projective Measurement. Outcome m.

Physical Z_1 pushes through $\mathrm{CX}_{0\to1}$ to $\mathbf{Z_0} \otimes \mathbf{Z_1}$.

Virtual $V = \mathrm{CX}_{1\to0}$ compresses it to Z_0. Measure Active q_0.

$\tilde{P}^{(t)} \leftarrow$ Updates $\mathbf{x}, \mathbf{z}$ tracking.

$A = \emptyset, \quad D = \{0, 1\} \quad (\mathbf{k} = \mathbf{0})$

$|\phi\rangle_A = [1]$ *(Array Halves!)*

$\gamma \leftarrow \gamma / \sqrt{p_m}$

Software Architecture

The "Aha!" Moment: Theorem 1

Look closely at the walkthrough. When did U_C update? When did the Pauli operators compress?

Theorem 1: Clifford Frame Determinism

The evolution of the Clifford Frame U_C and the dimensional bounds of the active set ($k_{\max}$) depend *exclusively* on the sequence of physical operations. They are strictly independent of probabilistic measurement outcomes or continuous non-Clifford amplitudes.

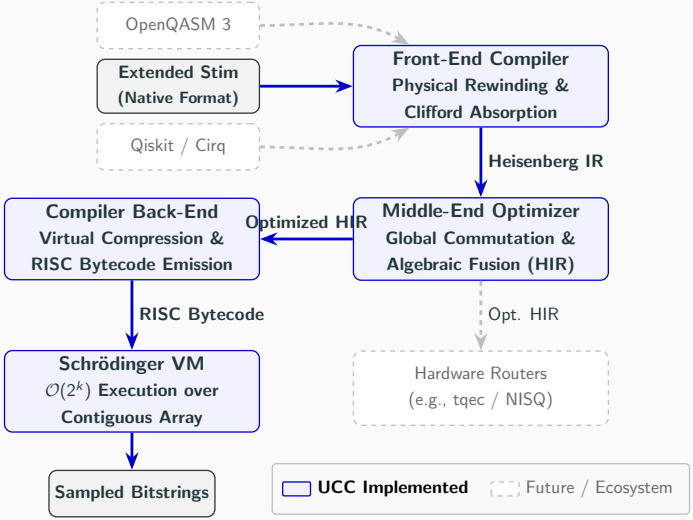
Consequence: We do not need to do heavy tableau math online. We cleanly split the simulation into an **Offline Multi-Level Compiler** and an **Online Virtual Machine**.

Decoupling Computational Complexity

1. **Multi-Level Compiler (Offline):**
   - Eats the $\mathcal{O}(Gn^2)$ topological matrix tracking.
   - Evaluates optimal Pauli Compressions to single virtual axes.
   - Computes global algebraic optimizations over the timeless **Heisenberg IR (HIR)**.

2. **The Schrödinger Virtual Machine (Online):**
   - Executes in strictly bounded $\mathcal{O}(2^{k_{\max}})$.
   - No dynamic memory allocation. Executes purely localized RISC bytecode.
   - *Note: The extreme C++ engineering making the VM fast (32-byte opcodes, branchless hardware popcounts) is a topic for a future deep-dive!*

UCC Architecture Diagram



Benchmarks & Results

Live Demo

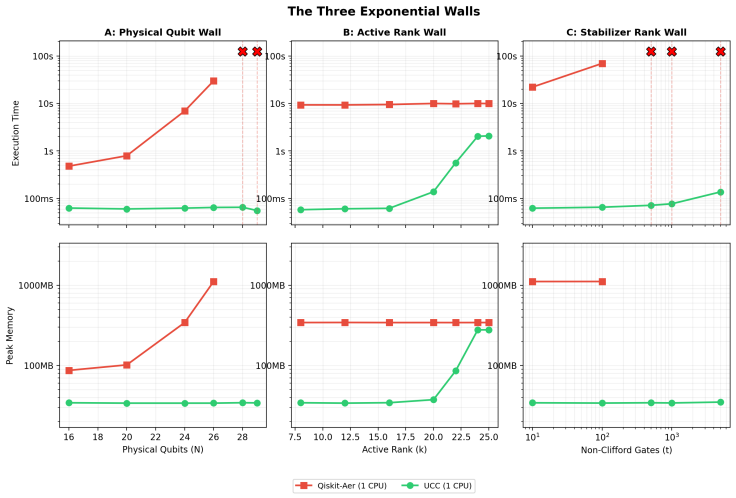
Manually driving compiler and evaluating active bounds

The "Three Walls" Benchmark

Each circuit has N physical qubits, k peak active non-Clifford virtual qubits, and t total T-gates. We isolate each axis on identical random Clifford+T circuits, comparing UCC against Qiskit-Aer (statevector) on a single CPU core:

- **Panel A — Physical Qubit Wall (N):** Fix k=12, t=20, sweep N. Qiskit scales as $\mathcal{O}(2^N)$; it times out at N=28 and OOMs at N=29. UCC remains flat at $\sim$60 ms.
- **Panel B — Active Rank Wall (k):** Fix N=24, t=40, sweep k. Qiskit is constant (always pays 2^{24}). UCC scales as $\mathcal{O}(2^k)$, revealing its honest exponential bound.
- **Panel C — Gate Depth Wall (t):** Fix N=26, k=12, sweep t. Both simulators scale linearly in t. *Planned:* adding `tsim` (stabilizer-rank) to Panel C will expose the $\mathcal{O}(2^{\alpha t})$ wall that UCC avoids.

Three Walls: UCC vs. Qiskit-Aer



Dashed/✕ = timeout (120 s) / OOM (6.5 GB). Single-threaded, Intel Xeon 8259CL, 7.2 GB RAM.

Scaling Up: Magic State Cultivation

Goal: Evaluate Magic State Cultivation (MSC) protocols at scale.

Phase 1 Validated ($d = 5$ Cultivation Step):

- The SOFT simulator (Li et al. 2025) previously required a 16-GPU cluster to process online dynamic tableaus for this 42-qubit circuit.
- By utilizing *Dense Survivor Sampling* and fast-fail opcodes, **UCC accurately matched the results on a single physical CPU core**.
- **Estimated Trillion-Shot Cost:** $\sim$\$500 and 29 hours on an AWS spot cluster to fully map this specific $d = 5$ protocol curve.

Next Target: We are actively estimating the runtime and cost required to execute the *full* 463-qubit ($d = 7$) end-to-end cultivation protocol.

Next Steps & Discussion

Feedback & Discussion

1. **Project Scope:** Should this Multi-Level VM architecture fully replace the old UCC (bridging/routing), or sit alongside it as a specialized engine?

2. **Publication Strategy:** What are the most critical benchmarks to include in the initial paper? (e.g., 3-Walls, FastTODD optimization, MSC validation).

3. **Narrative Framing:** Does positioning UCC as "A Multi-Level Compiler & Execution Engine for the Early Fault Tolerance Era" resonate with the foundation's vision?

Thank You!

Questions?